

MODUL PRAKTIK/PRAKTIKUM

IF451 – Advance Web Programming

PROGRAM SARJANA S1

FAKULTAS TEKNIK DAN INFORMATIKA

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA

Gedung B Lantai 5, Kampus UMN

Jl. Scientia Boulevard, Gading Serpong, Tangerang-15811 Indonesia

Telp: +62-21.5422.0808, web: <http://www.umn.ac.id>



Modul Week 8 – Introduction to Express.js

I. PENDAHULUAN

Express.js adalah **framework web untuk Node.js** yang mempermudah dalam membangun aplikasi web. Express menyediakan cara sederhana untuk **menangani routing, middleware, request/response**, serta mendukung **integrasi dengan database dan template engine**. Dengan Express, kode yang sebelumnya panjang (misalnya membuat server dan routing manual di Node.js) dapat ditulis jauh lebih ringkas.

II. CAPAIAN PRAKTIKUM

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Menginstal dan menggunakan Express.js.
2. Membuat server sederhana dengan Express.
3. Menangani routing dasar dengan metode GET dan POST.
4. Mengirimkan respon dalam bentuk HTML maupun JSON.

III. LANGKAH INSTALASI EXPRESS.JS

1. Buatlah folder baru dengan nama **"NIM_Week8"**.
2. Kemudian, lakukan inisialisasi project node.js (pastikan berada pada folder yang dibuat tadi).

```
93655_Week8>npm init -y
```

3. Lakukan instalasi Express.js, dengan command berikut:

```
npm install express
```

```
\93655_Week8>npm install express
```

IV. LANGKAH PRAKTIKUM

A. Membuat Server Express

1. Buatlah file baru dengan nama **app.js**.

```
JS app.js > ...
1  const express = require("express");
2  const app = express();
3  const port = 3000;
4
5  app.get("/", (req, res) => {
6    res.send("Selamat datang di Sistem Reservasi Ruang Rapat!");
7  });
8
9  app.listen(port, () => {
10    console.log(`Server berjalan di http://localhost:${port}`);
11  });
12
```

2. Jalankan dengan nodemon app.js. Lalu buka <http://localhost:3000>.

B. Routing Dasar (GET)

1. Setelah membuat server pada Express, selanjutnya adalah membuat routing dasar (GET) pada Express.
2. Modifikasi app.js dengan menambahkan routing seperti berikut:

```
9  app.get("/ruangan", (req, res) => {
10    res.send("Daftar ruang rapat: Ruang A, Ruang B, Ruang C");
11  });
12
13  app.get("/kontak", (req, res) => {
14    res.send("Hubungi admin di email: admin@kampus.ac.id");
15  });
```

3. Kemudian akses <http://localhost:3000/ruangan> dan <http://localhost:3000/kontak>

C. Menggunakan Parameter (GET)

1. Query Parameter:

Kembali menambahkan kode berikut pada **app.js**.

```
17 app.get("/reservasi", (req, res) => {
18   const nama = req.query.nama || "Tamu";
19   const ruang = req.query.ruang || "belum dipilih";
20   res.send(`Halo ${nama}, Anda memilih ${ruang} untuk reservasi.`);
21 });
22
```

Keterangan:

- req.query menangkap parameter **setelah tanda ?** pada URL.
- Contoh:
<http://localhost:3000/reservasi?nama=GregoriusJordan&ruang=Ruang+A>
- Hasil pada browser:

Halo GregoriusJordan, Anda memilih Ruang A untuk reservasi.

2. Route Parameter:

Tambahkan kode berikut untuk route parameter pada **app.js**

```
23 app.get("/ruangan/:id", (req, res) => {
24   const id = req.params.id;
25   res.send(`Detail Ruangan dengan ID: ${id}`);
26 });
27
```

Keterangan:

- req.params menangkap nilai dari URL yang ditulis **dengan :**
- Contoh: <http://localhost:3000/ruangan/101>

D. Membuat Form Input (POST)

1. Buatlah file baru dengan nama **form.html** dan salin kode dari bawah.

```
1 <h2>Form Reservasi</h2>
2 <form method="POST" action="/submit">
3   Nama: <input type="text" name="txtNama"><br>
4   Ruang: <input type="text" name="txtRuang"><br>
5   Tanggal: <input type="date" name="txtTanggal"><br>
6   <button type="submit">Kirim</button>
7 </form>
8
```

2. Kemudian, tambahkan kode berikut ini pada **app.js**.

```
29 const path = require("path");
30 app.use(express.urlencoded({ extended: true }));
31
32 app.get("/form", (req, res) => {
33   res.sendFile(path.join(__dirname, "form.html"));
34 });
35
36 app.post("/submit", (req, res) => {
37   const { txtNama, txtRuang, txtTanggal } = req.body;
38   res.send(`
39     <h3>Reservasi diterima:</h3>
40     Nama: ${txtNama}<br>
41     Ruang: ${txtRuang}<br>
42     Tanggal: ${txtTanggal}
43   `);
44 });
45
```

3. Kemudian coba jalankan <http://localhost:3000/form>. Data yang diisi melalui form akan dikirimkan dengan POST Method.

E. JSON Response

1. Untuk mengubah parameter menjadi JSON, cukup mengganti menjadi **res.json**.
2. Modifikasi app.js seperti kode berikut.

```
app.get("/api/ruangan", (req, res) => {
  res.json([
    { id: 1, nama: "Ruang A", kapasitas: 20 },
    { id: 2, nama: "Ruang B", kapasitas: 15 },
    { id: 3, nama: "Ruang C", kapasitas: 10 }
  ]);
});
```

3. Kemudian, akses <http://localhost:3000/api/ruangan>. Hasilnya akan berupa JSON.

TUGAS

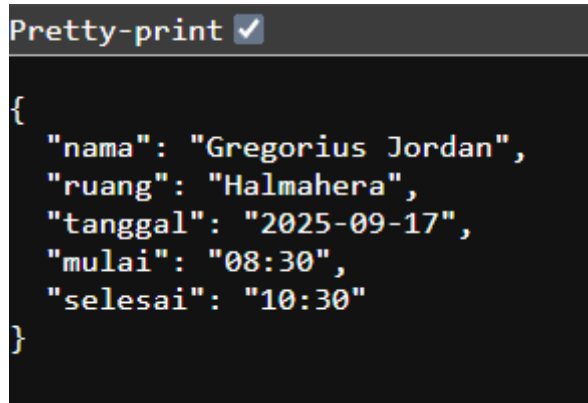
1. Tambahkan field **Waktu Mulai** dan **Waktu Selesai** di form reservasi.
2. Buat endpoint **/api/reservasi** yang menerima data POST form dalam format JSON.
Petunjuk:
 - Endpoint = route baru di server yang bisa diakses aplikasi.
 - /api/reservasi adalah endpoint khusus API.

- Bedanya dengan /submit (yang balas HTML), endpoint ini harus membalas dalam bentuk JSON.

Sehingga alurnya menjadi:

1. Mengisi form → klik Kirim.
2. Server menerima data (Nama, Ruang, Tanggal, Waktu Mulai, Waktu Selesai).
3. Server kembalikan data itu dalam format JSON.

Contoh hasil:



The image shows a screenshot of a web application's response in a 'Pretty-print' tool. The tool's header is dark grey with the text 'Pretty-print' in white and a checked checkbox to its right. The main area is black with white text displaying a JSON object. The JSON object has five key-value pairs: 'nama' with value 'Gregorius Jordan', 'ruang' with value 'Halmahera', 'tanggal' with value '2025-09-17', 'mulai' with value '08:30', and 'selesai' with value '10:30'. The JSON is formatted with indentation for readability.

```
{  
  "nama": "Gregorius Jordan",  
  "ruang": "Halmahera",  
  "tanggal": "2025-09-17",  
  "mulai": "08:30",  
  "selesai": "10:30"  
}
```